



TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES WEB

Departamento de Informática

PROYECTO VANLIFE



Manual Técnico

Autora: Julia San José León
Curso Académico: 2025-2026

ÍNDICE

1. Introducción	3
2. Arquitectura de la aplicación	3
2.1 Frontend	4
2.1.1 Tecnologías usadas	4
2.1.2 Justificación de la elección	4
2.1.3 Entorno de desarrollo	5
2.2 Backend	5
2.2.1 Tecnologías usadas	5
2.2.2 Justificación de la elección	5
2.2.3 Entorno de desarrollo	5
3. Documentación técnica	6
3.1 Análisis	6
3.1.1 Roles principales	6
3.1.2 Requisitos funcionales	6
3.1.3 Requisitos no funcionales	7
3.2 Desarrollo	8
3.2.1 Modelo de base de datos (Backend)	8
3.2.2 Diccionario y descripción de tablas	8
3.2.3 Justificación del esquema gráfico e integración con Supabase Auth	9
3.2.4 Políticas de Seguridad de Filas (Row Level Security - RLS)	10
3.2.5 Justificación Técnico-arquitectónica del diseño	10
3.2.6 Diagrama de casos de uso	12
3.2.7 Flujo de Navegación	13
3.3 Pruebas realizadas	14
4. Proceso de despliegue	15
4.1 Preparación del entorno	15
4.2 Construcción del proyecto	16
4.3 Despliegue en producción en vercel	16
4.4 Configuración final en Supabase	16
5. Propuestas de mejoras	17
6. Bibliografía	18

1. Introducción

El presente documento constituye el Manual Técnico de VanLife, una aplicación web desarrollada como Proyecto de fin de grado del ciclo formativo de Grado Superior de Desarrollo de Aplicaciones Web.

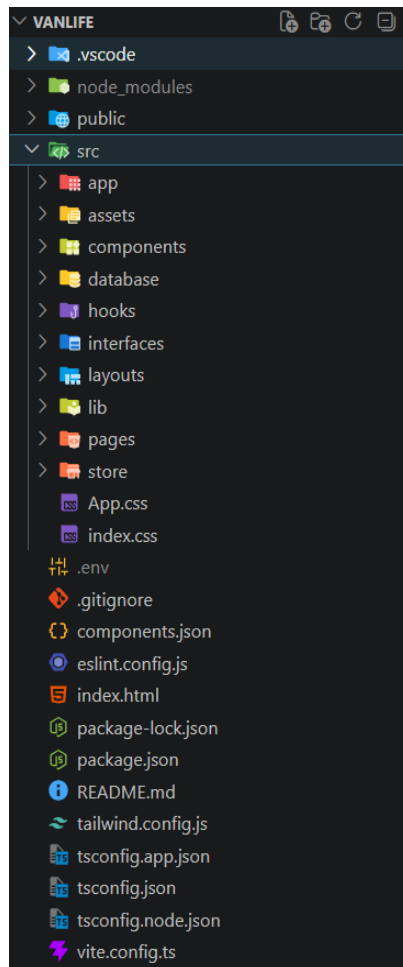
Este manual está dirigido a perfiles técnicos, administradores de sistemas y futuros desarrolladores que necesiten comprender la arquitectura, el código fuente y las decisiones de diseño implementadas en el aplicativo. En él se detallan las tecnologías utilizadas tanto en el lado del cliente frontend como en el servidor backend, los requisitos del entorno de desarrollo, el modelo de datos y la configuración necesaria para su despliegue en producción.

El objetivo de VanLife es proporcionar una herramienta de asistencia y geolocalización de servicios esenciales para la comunidad camper, apoyándose en una arquitectura moderna basada en componentes, mapas interactivos y servicios en la nube.

2. Arquitectura de la aplicación

La aplicación sigue una arquitectura cliente-servidor fuertemente desacoplada. El cliente consume servicios tanto del backend propio Supabase como de APIs externas Overpass API de OpenStreetMap para garantizar la obtención de datos geoespaciales en tiempo real.

En la siguiente imagen encontramos la estructura del proyecto:



2.1 Frontend

2.1.1 Tecnologías usadas

- **Lógica y Estructura:** React (v18), TypeScript y Vite.
- **Estilos y UI:** Tailwind CSS, Shadcn UI y Lucide React.
- **Geolocalización:** Leaflet y React-Leaflet.
- **Gestión de Estado y Rutas:** Zustand y React Router DOM.

2.1.2 Justificación de la elección

Se ha optado por React combinado con TypeScript para garantizar un desarrollo modular, basado en componentes reutilizables, y fuertemente tipado, lo que reduce drásticamente los errores en tiempo de ejecución. Vite reemplaza a Webpack como empaquetador debido a su velocidad de compilación en frío y soporte nativo para módulos ES. Para la interfaz, Tailwind CSS junto a Shadcn UI permite crear un diseño 100% responsive y accesible sin depender de pesados frameworks de UI, manteniendo el control total sobre el código de los componentes. Por último, Leaflet

es el estándar de código abierto para mapas interactivos, siendo la solución más ligera y compatible para renderizar las coordenadas obtenidas de OpenStreetMap.

2.1.3 Entorno de desarrollo

- **IDE:** Visual Studio Code con extensiones de formateo (Prettier, ESLint).
- **Gestor de paquetes:** npm (Node Package Manager).
- **Control de versiones y Despliegue:** Git, GitHub y despliegue continuo (CI/CD) en Vercel.

2.2 Backend

2.2.1 Tecnologías usadas

- **BaaS (Backend as a Service):** Supabase.
- **Base de datos:** PostgreSQL.
- **Autenticación:** Supabase Auth (JWT).
- **Almacenamiento:** Supabase Storage.
- **APIs Externas:** Overpass API (OpenStreetMap).

2.2.2 Justificación de la elección

Se ha descartado el desarrollo de un backend monolítico tradicional en favor de Supabase, una alternativa open-source a Firebase basada en PostgreSQL. Esta decisión permite agilizar el desarrollo (reduciendo el *time-to-market*) manteniendo la potencia de las bases de datos relacionales y la seguridad mediante Row Level Security (RLS), lo que impide que un usuario manipule los datos (rutas o lugares favoritos) de otro. Para la obtención de puntos de interés (gasolineras, áreas de pernocta, agua potable), se utiliza la Overpass API, lo que exime a la aplicación de mantener una base de datos geoespacial masiva propia, consultando directamente los servidores actualizados de OpenStreetMap de forma asíncrona.

2.2.3 Entorno de desarrollo

- **Gestor de base de datos:** Supabase Dashboard (interfaz gráfica en la nube).
- **Testing de APIs:** Postman o Insomnia para depurar las consultas (Query Language) de Overpass antes de integrarlas en las funciones de React.

3. Documentación técnica

3.1 Análisis

En esta sección se resumen los principales requisitos funcionales y no funcionales que han guiado el desarrollo de VanLife.

3.1.1 Roles principales

El control de acceso está gestionado por el flujo de autenticación de Supabase y validado en el frontend mediante Protected Routes de React Router.

- **Invitado:** Acceso público de lectura Landing page y previsualización del mapa.
- **Usuario registrado:** Genera un token JWT que le permite operaciones CRUD sobre sus rutas personales, perfil de vehículo y colección de sitios favoritos.
- **Administrador:** Cuenta con privilegios a nivel de base de datos para la moderación de contenido y la gestión global de usuarios.

3.1.2 Requisitos funcionales

Son las funciones o características que el sistema debe ofrecer al usuario.

1. Gestión de usuarios y perfiles

- **RF1.** El sistema permitirá el registro y autenticación segura de los usuarios utilizando Supabase Auth.
- **RF2.** El usuario podrá editar su perfil, incluyendo información clave como el tipo de vehículo, para adaptar las rutas y recomendaciones.
- **RF3.** El usuario podrá cerrar sesión de forma segura.

2. Mapa interactivo y ecosistema de datos

- **RF4.** La aplicación mostrará un mapa interactivo mediante Leaflet que geolocalizará puntos de interés en tiempo real.
- **RF5.** El sistema se integrará con OpenStreetMap a través de la Overpass API para obtener información actualizada del entorno.
- **RF6.** El usuario podrá filtrar los *Lugares Principales* mostrados en el mapa (Área AC, Camping, Parking, Gasolinera, Naturaleza, Monumentos).
- **RF7.** El usuario podrá realizar una búsqueda avanzada filtrando por *Servicios Adicionales* específicos (Agua potable, Duchas, Baños, Wifi, Basuras, Puntos EV, Admite mascotas, Farmacias).

3. Rutas y sitios favoritos

- **RF8.** El usuario registrado podrá utilizar React Router para la gestión y creación de rutas optimizadas.

- **RF9.** El usuario podrá guardar puntos de interés en su lista personal de favoritos (sitios_favoritos) para futuras consultas.
4. Comunidad y valoraciones
- **RF10.** Los usuarios podrán compartir información relevante, comentarios y valoraciones con la comunidad camper (rutas_guardadas).
5. Administración y configuración
- **RF11.** Existirá un panel de administración para gestionar usuarios, así como moderar y eliminar contenido inapropiado.

3.1.3 Requisitos no funcionales

Son las características de calidad del sistema, no relacionadas directamente con las funciones.

1. Usabilidad y diseño

- **RNF1.** La aplicación tendrá un diseño totalmente responsive (adaptable a cualquier pantalla) utilizando Tailwind CSS.
- **RNF2.** La interfaz será limpia, atractiva y coherente, construida mediante componentes modulares y reutilizables de Shadcn UI.

2. Rendimiento y arquitectura

- **RNF3.** El sistema utilizará Vite como herramienta de construcción para asegurar una carga rápida y tiempos de recarga optimizados en desarrollo.
- **RNF4.** La gestión del estado global de la aplicación se realizará de forma eficiente mediante Zustand, evitando renderizados innecesarios.

3. Seguridad

- **RNF5.** La protección de los datos estará garantizada mediante Políticas de Seguridad de Filas (RLS) en la base de datos PostgreSQL de Supabase.
- **RNF6.** Se aplicará un sistema de roles estricto (Invitado, Registrado y Administrador) para proteger las rutas privadas y las peticiones a la base de datos.

4. Mantenibilidad, escalabilidad y despliegue

- **RNF7.** El código fuente estará alojado en repositorios con control de versiones (Git) y segmentado de manera modular.

- **RNF8.** El sistema será altamente escalable al utilizar una arquitectura BaaS (Backend as a Service) con Supabase para la base de datos y el almacenamiento (Storage).

5. Disponibilidad

- **RNF9.** La aplicación debe estar disponible las 24 horas del día. Al estar desplegada en la infraestructura en la nube de Vercel y utilizar Supabase, se garantiza una alta disponibilidad (uptime mínimo del 99%).

3.2 Desarrollo

El desarrollo de VanLife se ha llevado a cabo utilizando un enfoque modular, separando claramente la lógica de negocio, la gestión del estado y la interfaz de usuario.

La aplicación utiliza una base de datos PostgreSQL gestionada por Supabase. La estructura de tablas se ha diseñado teniendo en cuenta la integración directa con el sistema de autenticación Supabase Auth, lo cual influye en la forma en que se representan las relaciones dentro del modelo.

3.2.1 Modelo de base de datos (Backend)

La persistencia de los datos, la gestión de sesiones de usuario y las relaciones de la comunidad camper de VanLife se han implementado sobre un motor de base de datos relacional PostgreSQL, administrado de forma remota a través de la plataforma BaaS Supabase.

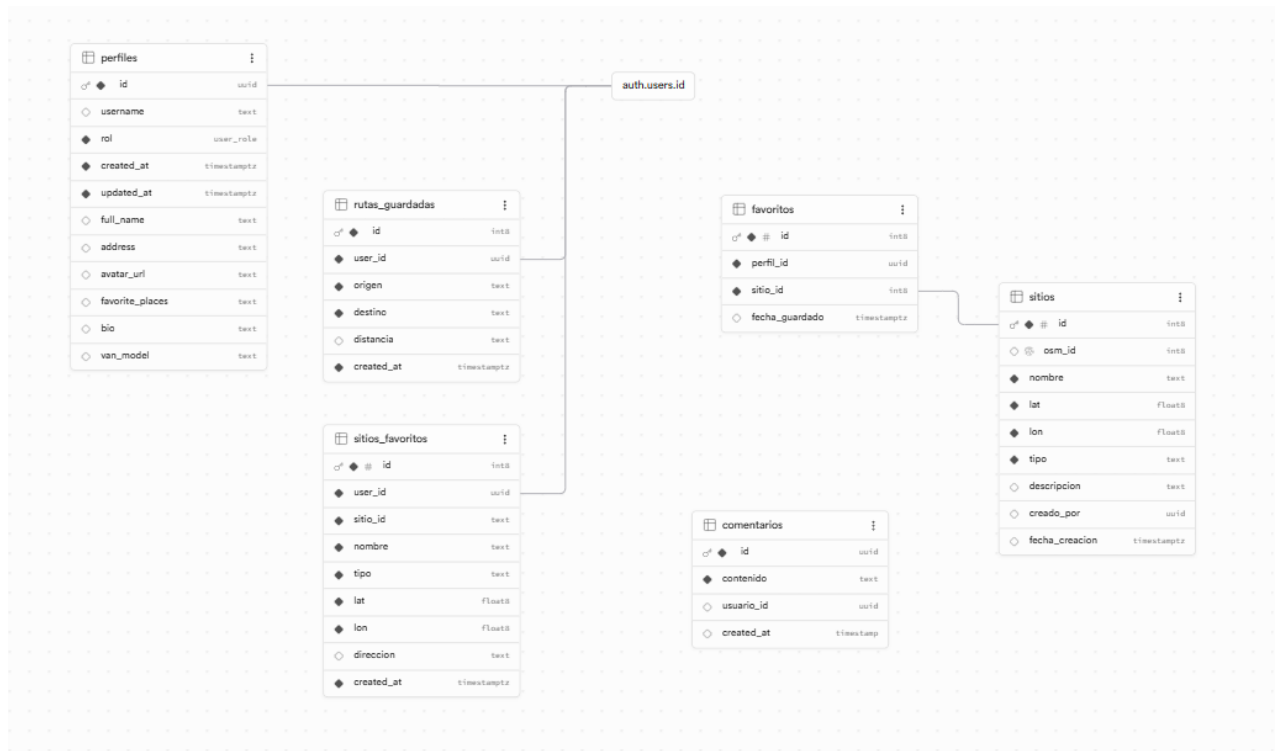
3.2.2 Diccionario y descripción de tablas

El esquema de datos público (public) está diseñado para asegurar la integridad referencial y optimizar las consultas del mapa interactivo:

- **perfiles:** Extiende la información de las cuentas de usuario. Almacena el nombre del viajero, su fotografía de avatar, biografía y el tipo de vehículo (camper, autocaravana, furgoneta, etc.), parámetro crítico para la personalización de las rutas.
- **sitios:** Almacena los puntos críticos y de pernocta creados específicamente por la comunidad (coordenadas geográficas de latitud/longitud, descripción pormenorizada, imágenes asociadas y bloques de servicios disponibles).
- **rutas_guardadas:** Registra los itinerarios personalizados planificados por los usuarios, así como el histórico de comentarios y valoraciones vertidos sobre la plataforma.

- **sitios_favoritos / favoritos:** Actúa como tabla puente relacional para resolver la lógica de muchos a muchos (N:M) entre los usuarios autenticados y sus lugares marcados de interés.

3.2.3 Justificación del esquema gráfico e integración con Supabase Auth



En esta vista del visualizador gráfico se aprecia que las tablas aparecen como entidades independientes y no muestran relaciones visuales explícitas entre sí. Esto puede resultar llamativo a primera vista y dar la errónea impresión de que falta cohesión o integridad referencial en el diseño del esquema de almacenamiento.

Sin embargo, este comportamiento es totalmente normal y esperado cuando se trabaja con Supabase Auth. Supabase aísla por motivos de seguridad el control de accesos en un esquema interno del sistema denominado `auth`, cuya tabla principal es `auth.users`. Esta tabla actúa como la entidad centralizada que contiene:

- El identificador único universal del usuario (UUID).
- Las credenciales de acceso seguras (email y hashes cifrados).

Dado que la herramienta de diagramación de Supabase renderiza de forma exclusiva las tablas alojadas en el esquema público (`public`), los enlaces de clave foránea (*Foreign Keys*) que cruzan la frontera hacia el esquema restringido `auth` no se proyectan visualmente mediante líneas continuas.

En la arquitectura de VanLife, la integridad relacional queda firmemente resuelta enlazando la tabla `perfiles` a `auth.users` mediante una clave primaria que hereda dicho UUID. A partir de este perfil público, se estructuran el resto de dependencias (claves foráneas en `sitios`, `sitios_favoritos` y `rutas_guardadas`), garantizando la consistencia interna de los datos y las cascadas de borrado directamente en el motor PostgreSQL, al margen de las limitaciones de representación de la interfaz gráfica.

3.2.4 Políticas de Seguridad de Filas (Row Level Security - RLS)

Para evitar que un usuario manipule o acceda a la información privada de otro, el motor de la base de datos se encuentra protegido mediante políticas RLS estrictas. Cualquier petición HTTP (SELECT, INSERT, UPDATE, DELETE) enviada desde el frontend es evaluada por PostgreSQL comprobando que el UUID del token JWT del usuario coincide de forma unívoca con el identificador de la fila que se intenta alterar.

3.2.5 Justificación Técnico-arquitectónica del diseño

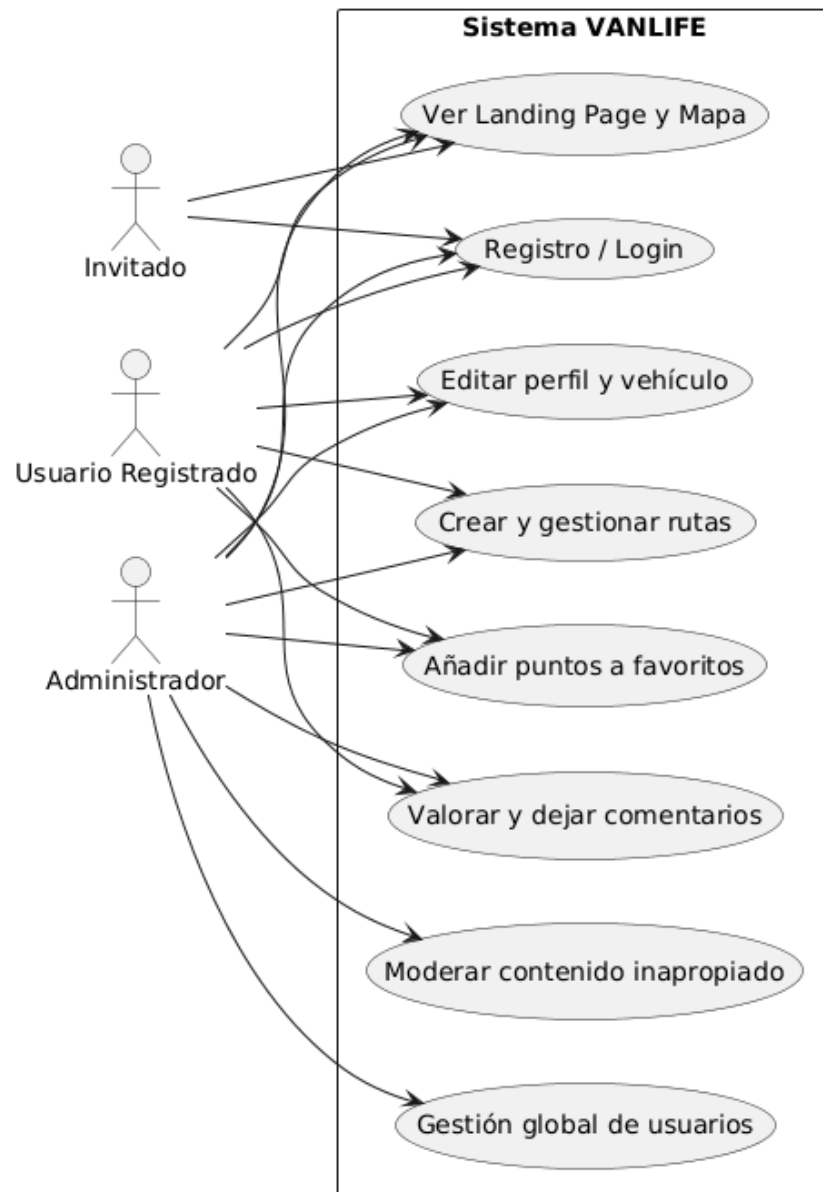
El uso del esquema protegido de Supabase y, específicamente, de `auth.users` como la tabla raíz de toda la arquitectura de datos de VanLife, ofrece ventajas críticas en términos de seguridad, rendimiento y mantenibilidad:

- **Integración natural con Supabase Auth:** Establece un único origen de verdad para la identidad de los usuarios campistas de la plataforma. Al delegar la gestión de sesiones basadas en tokens JWT y el cifrado de contraseñas a la infraestructura nativa de Supabase, se reduce la superficie de ataque y la complejidad del desarrollo backend.

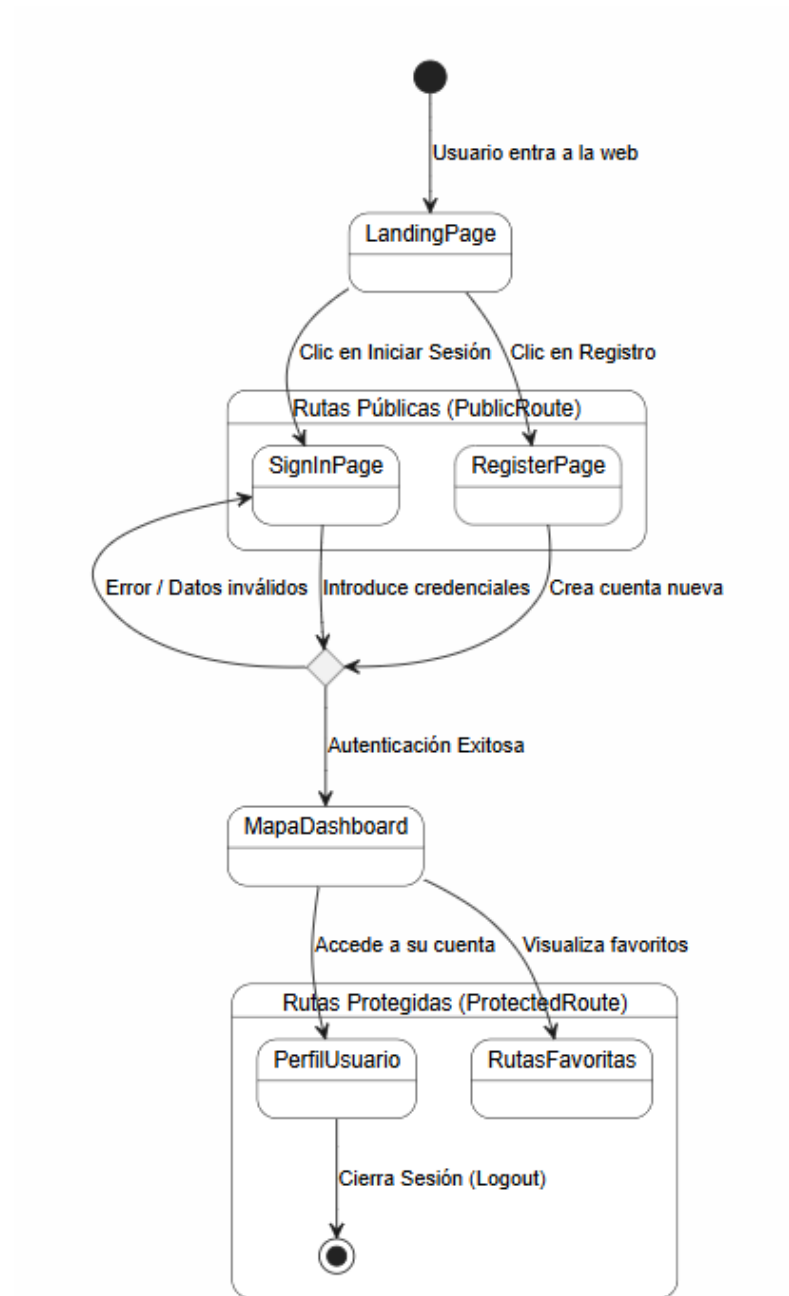
- **Máxima seguridad mediante RLS:** Las Políticas de Seguridad de Filas (Row Level Security) configuradas en PostgreSQL pueden interceptar de manera eficiente las consultas del frontend. El motor de la base de datos compara directamente la identidad empaquetada en el token del cliente (auth.uid()) con el campo usuario_id mapeado en las tablas del esquema público (como sitios_favoritos o rutas_guardadas), denegando inmediatamente cualquier petición de manipulación de datos no autorizada.
- **Minimización de la redundancia de datos:** Al desvincular por completo las credenciales estrictas de acceso de la información del dominio camper, se evita duplicar o propagar relaciones complejas y datos sensibles a lo largo del esquema public. La tabla perfiles actúa como un puente intermedio limpio que distribuye el identificador unívoco a los módulos funcionales requeridos.
- **Evolución y escalabilidad modular:** Cualquier cambio futuro o expansión en la estructura del perfil del viajero (por ejemplo, añadir campos para segmentar nuevas especificaciones técnicas de vehículos, preferencias de pernocta o configuraciones de i18n) se ejecuta de forma aislada sobre la tabla perfiles. Esto garantiza que las modificaciones no tengan un impacto colateral en el núcleo del sistema de autenticación ni quiebren las restricciones de integridad de las tablas dependientes (sitios, rutas_guardadas).

Este enfoque representa el estándar arquitectónico recomendado por Supabase para aplicaciones estructuradas bajo el paradigma Single Page Application (SPA), las cuales delegan el control de accesos, el almacenamiento de ficheros (Storage) y la persistencia relacional directamente en el proveedor de infraestructura en la nube.

3.2.6 Diagrama de casos de uso



3.2.7 Flujo de Navegación



3.3 Pruebas realizadas

Aunque no se ha implantado un framework de pruebas automatizadas completo (como Jest o Cypress), durante la fase de desarrollo y pre-producción se han llevado a cabo pruebas manuales sistemáticas y exhaustivas sobre las funcionalidades críticas del aplicativo:

Pruebas de autenticación (Registro, Login y Zustand)

- **Datos de entrada:**
 - *Caso válido:* Email correcto, contraseña segura y confirmación de contraseña coincidente en el registro.
 - *Casos inválidos:* Email vacío o con formato incorrecto, contraseñas menores a 6 caracteres o que no coinciden.
- **Resultado esperado:**
 - En los casos inválidos, la validación del formulario en el frontend bloquea el envío y muestra mensajes de error específicos al usuario.
 - En el caso válido, Supabase procesa la petición, devuelve un token de sesión, el estado global (Zustand) se actualiza y el usuario es redirigido automáticamente al mapa interactivo o a su perfil.

Pruebas de Integración con el Mapa (Leaflet y Overpass API)

- **Datos de entrada:**
 - *Caso válido:* El usuario selecciona un filtro del panel (ej. "Agua Potable" o "Área AC") con una conexión a internet estable.
 - *Casos límite:* El usuario realiza una búsqueda en una zona geográfica donde no existen servicios registrados en OpenStreetMap, o la API tarda en responder.
- **Resultado esperado:**
 - En el caso válido, la aplicación procesa la consulta asíncrona y renderiza correctamente los marcadores personalizados (<Marker>) en las coordenadas exactas del mapa.
 - En los casos límite, la aplicación gestiona la promesa vacía sin romperse, mostrando un mensaje de "No se encontraron resultados en esta área" o un estado de carga (loader) mientras espera la respuesta.

Pruebas de persistencia y seguridad de datos (Favoritos y RLS)

- **Datos de entrada:**
 - *Caso de éxito:* Un usuario autenticado hace clic en el botón de "Añadir a Favoritos" en un punto de pernocta.
 - *Caso de fallo de seguridad:* Un usuario intenta forzar (mediante herramientas de desarrollador) el guardado de un favorito con el ID de otro usuario, o intenta guardar sin estar logueado.

- **Resultado esperado:**

- El usuario autenticado ve cómo el icono cambia de estado y el lugar se guarda correctamente en la tabla `sitios_favoritos` de su perfil.
- Las Políticas de Seguridad de Filas (RLS) en PostgreSQL interceptan y bloquean la petición manipulada, devolviendo un error 403 (Forbidden) y protegiendo la integridad de la base de datos.
- El usuario no logueado es redirigido inmediatamente a la pantalla de Login (gracias al componente `ProtectedRoute` de React Router).

Pruebas de diseño responsivo y accesibilidad

- Se ha verificado la correcta visualización de la plataforma (Landing, Mapa, Formularios) simulando diferentes resoluciones de pantalla (móvil, tablet, escritorio) a través de las DevTools del navegador.
- **Resultado esperado:** El diseño basado en Tailwind CSS colapsa los menús de navegación en un menú hamburguesa en dispositivos móviles y adapta las tarjetas de los lugares de pernocta sin que se desborde el contenido ni se pierda usabilidad.

4. Proceso de despliegue

El despliegue de VanLife se ha diseñado siguiendo un modelo de Integración y Despliegue Continuo (CI/CD) apoyado en repositorios Git, lo que permite que cada actualización en el código fuente se refleje de manera ágil en el entorno de producción. El frontend se aloja en la plataforma Vercel, mientras que el backend y la base de datos residen en la nube de Supabase.

4.1 Preparación del entorno

Para que la aplicación funcione correctamente tanto en desarrollo como en producción, es estrictamente necesario configurar las variables de entorno. Estas credenciales conectan el cliente de React con los servidores de Supabase.

En el entorno local, se requiere un archivo `.env` en la raíz del proyecto con la siguiente estructura:

- `VITE_SUPABASE_URL`: URL del proyecto asignada en el dashboard de Supabase.
- `VITE_SUPABASE_ANON_KEY`: Clave pública (API Key) para realizar peticiones seguras desde el cliente.

4.2 Construcción del proyecto

Dado que React y TypeScript no pueden ser interpretados directamente por el navegador, es necesario compilar el código. Utilizando Vite como empaquetador (bundler), se ejecuta el siguiente comando en la terminal:

- `npm run build`

Este proceso transpile el código TypeScript a JavaScript puro, optimiza los *assets* (como imágenes CSS y fuentes) y genera una carpeta llamada *dist* que contiene los archivos estáticos listos y minificados para subir a producción.

4.3 Despliegue en producción en vercel

El alojamiento web se realiza en Vercel por su compatibilidad nativa e integración perfecta con proyectos creados con Vite y React. Los pasos llevados a cabo para el despliegue son:

1. **Vinculación del repositorio:** Se conecta la cuenta de Vercel con el repositorio de GitHub donde está alojado el código fuente de VanLife (jsanjose101/VanLife).
2. **Configuración del framework:** Vercel detecta automáticamente que el proyecto utiliza Vite. Se verifican los comandos de compilación (`npm run build`) y el directorio de salida (*dist*).
3. **Inyección de variables de entorno:** En el panel de configuración del proyecto en Vercel (sección *Environment Variables*), se introducen las claves de Supabase (`VITE_SUPABASE_URL` y `VITE_SUPABASE_ANON_KEY`). Este paso es vital, ya que el archivo `.env` local nunca se sube a GitHub por motivos de seguridad (está incluido en el `.gitignore`).
4. **Despliegue y URL pública:** Al confirmar la configuración, Vercel compila el proyecto en sus propios servidores y genera un dominio público con certificado SSL (HTTPS) accesible para todos los usuarios. A partir de este momento, cualquier push realizado a la rama principal (*main/master*) en GitHub desencadenará un redesplicue automático.

4.4 Configuración final en Supabase

Una vez que el frontend está alojado en Vercel y se cuenta con una URL pública definitiva, es necesario realizar un último ajuste de seguridad en el backend:

- Se accede al Dashboard de Supabase, al apartado de *Authentication* -> *URL Configuration*.

- Se añade el dominio público generado por Vercel (ej. <https://vanlife.vercel.app>) a la lista de Site URL y Redirect URLs permitidas. De lo contrario, el sistema de autenticación bloquearía los inicios de sesión por políticas de seguridad (CORS).

5. Propuestas de mejoras

Aunque la versión actual de VanLife cumple con éxito todos los requisitos funcionales y técnicos definidos en la fase de análisis, el ciclo de vida del software permite una evolución constante. De cara a futuras iteraciones y para escalar el proyecto comercialmente, se proponen las siguientes líneas de mejora:

- **Sistema avanzado de comunidad y reseñas:** Implementar un módulo social donde los usuarios no solo guarden rutas, sino que puedan dejar comentarios, valoraciones (1 a 5 estrellas) y fotografías recientes en los puntos de pernocta. Esto fomentará el crecimiento orgánico de la comunidad, permitiendo que los propios viajeros validen el estado real de los servicios (por ejemplo, alertando si un punto de agua potable se encuentra fuera de servicio).
- **Soporte offline mediante PWA (Progressive web app):** Dado que el público objetivo suele viajar a zonas de montaña o rurales con escasa cobertura móvil, la integración de service workers permitiría descargar mapas de zonas específicas y acceder a los lugares favoritos guardados sin necesidad de conexión a internet.
- **Integración de API meteorológica:** Añadir el consumo de una API externa del clima (como OpenWeatherMap). Al seleccionar un punto en el mapa interactivo, el sistema mostraría la previsión meteorológica para esa noche, un dato crítico para los viajeros que duermen en vehículos.

6. Bibliografía

A continuación, se recogen las principales fuentes técnicas consultadas durante el desarrollo del proyecto y la elaboración de este manual:

- Documentación oficial de React: conceptos básicos, hooks y manejo del DOM virtual. <https://react.dev/>
- Documentación oficial de TypeScript: tipado estático e interfaces. <https://www.typescriptlang.org/>
- Documentación oficial de Vite: configuración y despliegue de SPAs. <https://vitejs.dev/>
- Documentación oficial de TailwindCSS: utilidades y diseño responsive. <https://tailwindcss.com/>
- Componentes modulares de Shadcn UI. <https://ui.shadcn.com/>
- Documentación oficial de Supabase: bases de datos, Auth, Storage y RLS. <https://supabase.com/docs>
- OpenStreetMap y manual de consultas de Overpass API: https://wiki.openstreetmap.org/wiki/Overpass_API
- Documentación de Vercel para el despliegue de aplicaciones frontend. <https://vercel.com/docs>